
Advanced Programming

ÁRPÁD TAMÁSI · ELTE FACULTY OF SOCIAL SCIENCES

SESSION 1 — 8 SEPTEMBER 2026



What changed
by early 2026

Pieter Levels (@levelsio) · 22 February 2025

- A browser flight simulator
- Ad space inside the game: airships, billboards
- \$87,000 a month by week three

I've never ever made a game before and just made my own flight simulator 100% with Cursor in I'd say 3 hours by just telling it what I wanted

FastRender

- Wilson Lin, a Cursor research project
- **Three weeks**, at peak ~2,000 parallel agents, ~30,000 commits
- **3 million lines of Rust**, roughly 110 person-years of work (*independent audit: SIG*)
- Its own HTML parser, CSS engine and JavaScript VM
- Loads Wikipedia and CNN
- [GitHub repository](#)

A C compiler in two weeks

- Anthropic, February 2026 · 16 Claude agents in parallel
- 100,000 lines, compiles Linux, **99%** on the GCC test suite

The same thing until now: GCC since 1984, clang since 2007 — decades, with teams.

How the tools changed

HOW PEOPLE WORKED

2021–2022

completion, one line at a time

2023–2024

chat, generated snippets

2025

agents read the repository, edit files, run tests, open pull requests

early 2026

delegating longer tasks, supervising several agents at once

Three claims to put on the board

1. The cost of **producing** code has fallen
2. The value of **goal, context and verification** has risen
3. Quality is established by **measurement**, not by impression

Where it breaks

Maintainability

FastRender, independent audit (SIG) on the public codebase:

- Maintainability **1.3 / 5** · architecture **2.1 / 5**
- The **least maintainable** 5% of systems on the market
- *benchmark: 400 billion lines of code — a new system is expected to reach 4 stars*

What others measured on it:

- **88%** of builds fail
- Redundant code: Ladybird and Servo **do more** in the same number of lines

Moltbook

A social network for AI agents, February 2026. It spread within days. The founder: “*I did not write a single line of code.*”

Three days after launch, Wiz researchers opened its database from a browser — readable and writable:

- **1.5 million** access keys
- 35,000 e-mail addresses, and every private message

Nobody said the data should stay private, nobody checked — **and the model did not raise it either.**

Faster?

16 experienced developers · 246 real tasks in their own repositories · randomly assigned whether AI could be used

	HOW MUCH FASTER?
What the developers predicted	+24%
What they believed afterwards	+20%
What was measured	−19%

(METR, July 2025. Large, old codebases the developers had known for years. A snapshot — but their impression was wrong all the same.)

Verifiability

Why the C compiler worked

- The C standard is 40 years old
- The test suites of earlier compilers were available
- ~3,900 test files, each with ten compiler settings
- ~40,000 test runs

“It’s important that the task verifier is nearly perfect, otherwise Claude will solve the wrong problem.”

— *Nicholas Carlini, Anthropic*

How your project becomes verifiable

- There is no forty-year-old standard
- There are no tests
- This was a hard problem before too — and there is a worked method for it
- We will specify the task **methodically**, with the machine helping
- The machine knows the method and will help you follow it — **but you have to know the goal**

Who is in the room

Sixty seconds each

1. Your **name** and where you come from
2. What have you used **Python** for so far?
3. What would you like to be able to **build on your own** by the end of term?

Where the group starts

Hands up — who has:

1. their **own laptop** here, and permission to install on it
2. ever run a **.py** file from a terminal
3. ever made a **branch** in Git
4. used an **AI chat** for coding

The demo

Would you accept this number?

The survey report prints

3.84

for weighted mean
satisfaction.

What the agent did · what the human decided

THE AGENT

read the files

searched and changed code

wrote and ran a test

summarised the change

THE HUMAN

set the goal and the limits

chose the authoritative domain source

checked what the test means

accepted or rejected the diff

Three questions, all term

1. What exactly do we want the program to do?
2. How do we know that it does that?
3. What risk is left after the change?

The semester



Your own project, end to end

problem and project brief

- specification
- technical spike
- walking skeleton
- tests
- first useful version
- integration and debugging
- review and refactoring
- reproducible release
- presentation and defence

Twelve sessions

8 September

today

26 October · 2 November

no class — autumn break, then the free fortnight

December 12

end of term

Every phase brief states the artefact, the definition of done, the frame for AI use, and what to do when you get stuck.

What you get, and what it costs

- \$30 of LLM API credit each, individually
- The course adds a **cost cap**, offline **fixtures**, and an access **fallback** — so a missing key or an empty balance never blocks the homework
- Editor, Git and environment manager: **the course stack**, in Homework 0

Homework 0

Get your environment running

1. Install the **editor, Git and environment manager** from the setup package
2. Download the **setup package**
3. Run `check_setup`
4. Submit the **full terminal output**

Deadline: 48 hours before session 2 — so there is time to fix what breaks.

Exit ticket

1. What is your own goal for the end of term?
2. What is the biggest obstacle to setting up your environment?
3. What was the most important human decision in the demo?